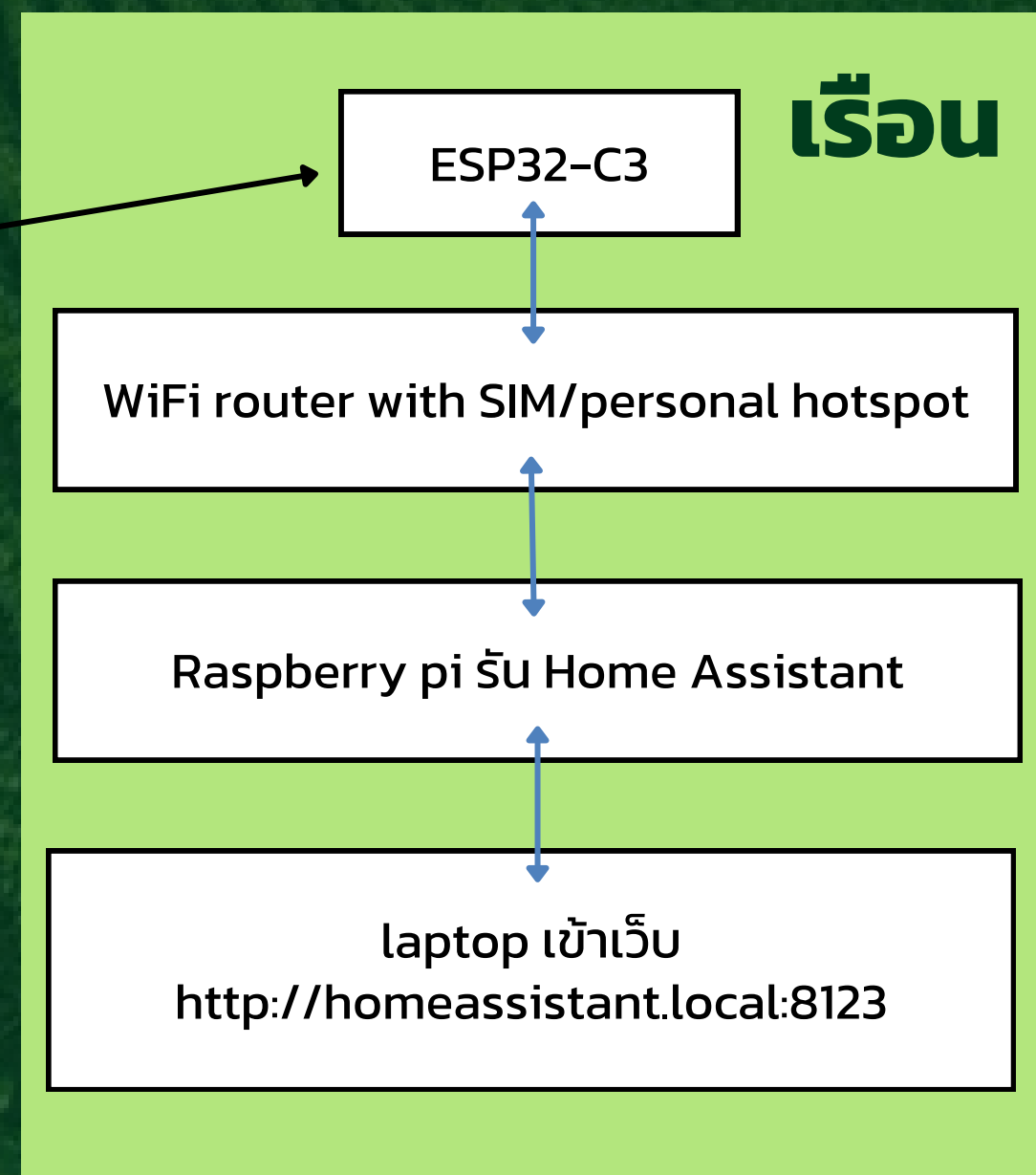
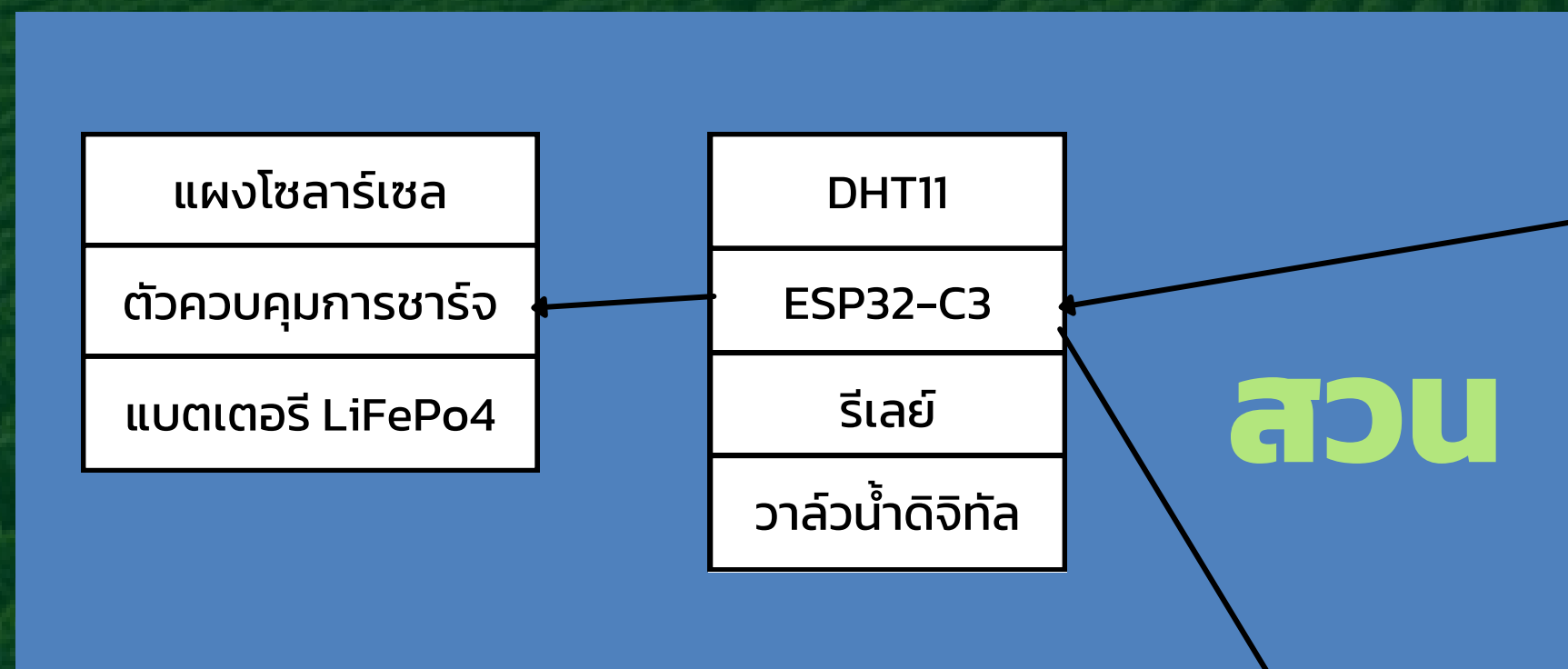


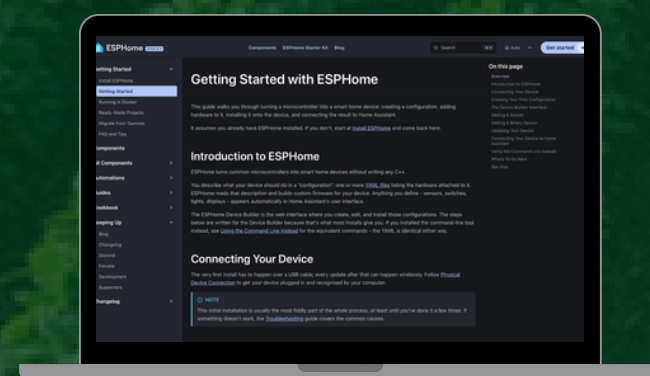
นวัตกรรมเกษตรอัจฉริยะ: AIOT สำหรับเกษตรกรแม่นยำ รุ่นที่ 1

25 ก.ย. 2569 – 29 ก.ย. 2569

การเชื่อมต่อ ระบบฟาร์มสู่ตลาด และแดนชบอรัต



ESPHome firmware

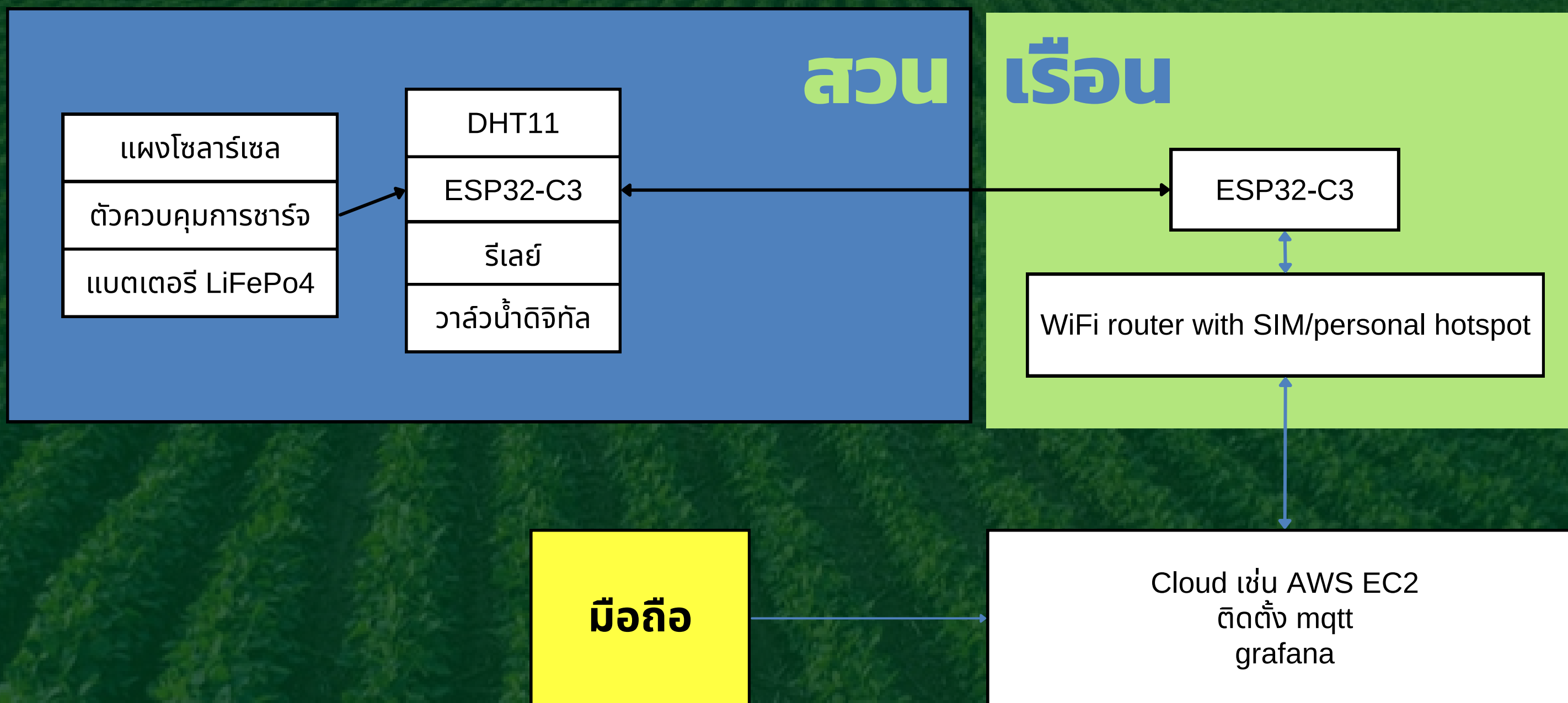


laptop เข้าเว็บ
<https://web.esphome.io/>

ดูสถานะแดชบอร์ด, กดปุ่มเปิดปิด, ลง ESPHome add-on
เพื่อเปิดหน้าต่าง ESPHome เขียน YAML

Home Assistant





โค้ด **MicroPython** สำหรับ บอร์ด 1 (ในสวน)

หน้าที่:

- อ่านค่า DHT22 ทุกๆ ช่วงเวลา (เช่น ทุก 5 นาที)
- ส่งค่าอุณหภูมิและความชื้นไปที่บอร์ด 2 ผ่าน ESP-NOW

คอยฟังคำสั่ง (เปิด/ปิด รีเลย์) จากบอร์ด 2 ผ่าน ESP-NOW แล้วควบคุมวาล์วน้ำ
คำแนะนำ: คุณต้องนำ MAC Address ของบอร์ด 2 มาใส่ใน BOARD_2_MAC

(หาได้จากการรัน `import network; wlan = network.WLAN(network.STA_IF); wlan.active(True); print(wlan.config('mac'))` บนบอร์ด 2)

บอร์ดในสวน

```
# main.py - บอร์ด 1 (ในสวน)
```

```
import network
```

```
import espnw
```

```
import dht
```

```
import machine
```

```
import time
```

```
import json
```

```
# --- กำหนดพิน ---
```

```
DHT_PIN = machine.Pin(4, machine.Pin.IN)    # พิน DHT22 (ปรับตามที่ต่อจริง)
```

```
RELAY_PIN = machine.Pin(5, machine.Pin.OUT)  # พิน Relay (Active HIGH หรือ LOW ตามประเภท  
รีเลย์)
```

```
RELAY_OFF = 0
```

```
# ปรับเป็น 1 ถ้าเป็น Active LOW
```

```
RELAY_ON = 1
```

```
# ปรับเป็น 0 ถ้าเป็น Active LOW
```

```
RELAY_PIN.value(RELAY_OFF)
```

```
sensor = dht.DHT22(DHT_PIN)
```

```
# --- ตั้งค่า ESP-NOW ---
```

```
wlan = network.WLAN(network.STA_IF)
```

```
wlan.active(True)
```

```
e = espnow.ESPNow()  
e.active(True)
```

```
# *** ใส่ MAC Address ของบอร์ด 2 (ในบ้าน) ***
```

```
BOARD_2_MAC = b'\xff\xff\xff\xff\xff\xff' # เช่น b'\x24\xd7\xeb\x12\x34\x56'
```

```
try:
```

```
    e.add_peer(BOARD_2_MAC)
```

```
except Exception:
```

```
    pass
```

```
last_send_time = 0
```

```
SEND_INTERVAL = 300 # ส่งค่าทุก 5 นาที (300 วินาที)
```

```
print("Board 1 (Garden) ready...")
```

```
while True:
```

```
    # 1. เช็คว่ามีคำสั่งจากบอร์ด 2 เข้ามาหรือไม่ (Non-blocking)
```

```
    host, msg = e.irecv(0)
```

```
    if msg:
```

```
        try:
```

```
            data = json.loads(msg.decode('utf-8'))
```

```
            if "cmd" in data:
```

```
                command = data["cmd"]
```

```
                if command == "WATER_ON":
```

```
                    RELAY_PIN.value(RELAY_ON)
```

```
                    print("Water Valve: OPEN")
```

```
                elif command == "WATER_OFF":
```

```
                    RELAY_PIN.value(RELAY_OFF)
```

```
                    print("Water Valve: CLOSED")
```

```
            except Exception as err:
```

```
                print("Error processing command:", err)
```

```
# 2. อ่านค่าเซนเซอร์และส่งให้กับบอร์ด 2 ตามช่วงเวลา
if time.time() - last_send_time >= SEND_INTERVAL:
    try:
        sensor.measure()
        temp = sensor.temperature()
        hum = sensor.humidity()

        payload = json.dumps({"temp": temp, "hum": hum})
        e.send(BOARD_2_MAC, payload)
        print(f"Sent sensor data: Temp={temp}C, Hum={hum}%")

        last_send_time = time.time()
    except Exception as err:
        print("Failed to read DHT22 or send via ESP-NOW:", err)

time.sleep_ms(100) # หน่วงเวลาสั้นๆ เพื่อไม่ให้ใช้ CPU เต็ม 100%
```

โค้ด **MicroPython** สำหรับ บอร์ด 2 (ในบ้าน)

หน้าที่:

- ต่อ WiFi เพื่อคุยกับ AWS EC2
- รับข้อมูลเซนเซอร์จากบอร์ด 1 แล้ว HTTP POST ขึ้น AWS EC2
- คอย Poll (สอบถาม) คำสั่งรดน้ำจาก AWS EC2 ทุกๆ 10-15 วินาที
- หาก EC2 สั่งให้เปิด/ปิดน้ำ จะส่งคำสั่งต่อไปให้บอร์ด 1 ผ่าน ESP-NOW



```
# main.py - บอร์ด 2 (ในบ้าน)
import network
import espnow
import urequests
import ujson
import time

# --- ตั้งค่า WiFi ---
WIFI_SSID = "YOUR_WIFI_NAME"
WIFI_PASS = "YOUR_WIFI_PASSWORD"

# *** ใส่ Public IP หรือ Domain ของ EC2 ***
EC2_URL = "http://YOUR_EC2_PUBLIC_IP:5000"

# *** ใส่ MAC Address ของบอร์ด 1 (ในสวน) ***
BOARD_1_MAC = b'\xff\xff\xff\xff\xff\xff'
```

```
def connect_wifi():
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    if not wlan.isconnected():
        print("Connecting to WiFi...")
        wlan.connect(WIFI_SSID, WIFI_PASS)
        while not wlan.isconnected():
            time.sleep(0.5)
    print("WiFi connected! IP:", wlan.ifconfig()[0])
```

```
connect_wifi()
```

```
# --- ตั้งค่า ESP-NOW ---
e = espnow.ESPNow()
e.active(True)
```

```
try:
    e.add_peer(BOARD_1_MAC)
except Exception:
    pass
```

```
last_poll_time = 0
POLL_INTERVAL = 15 # ดึงคำสั่งจาก EC2 ทุก 15 วินาที

print("Board 2 (Home Bridge) ready...")

while True:
    # 1. รับค่าจากบอร์ด 1 ผ่าน ESP-NOW แล้วยังส่งไปที่ EC2
    host, msg = e.irecv(0)
    if msg:
        try:
            payload_str = msg.decode('utf-8')
            sensor_data = ujson.loads(payload_str)
            print("Received from Board 1:", sensor_data)

            # ส่งไปเก็บที่ EC2
            res = urequests.post(
                f"{EC2_URL}/api/data",
                json=sensor_data,
                headers={'Content-Type': 'application/json'}
            )
```

```
res.close()
    print("Uploaded data to EC2 successfully")
except Exception as err:
    print("Error uploading data to EC2:", err)

# 2. คอยเช็คคำสั่งจาก EC2 (Polling)
if time.time() - last_poll_time >= POLL_INTERVAL:
    try:
        res = urequests.get(f"{EC2_URL}/api/command")
        if res.status_code == 200:
            cmd_data = res.json()
            cmd = cmd_data.get("cmd")

            # หากมีคำสั่งค้างอยู่ (เช่น WATER_ON หรือ WATER_OFF)
            if cmd in ["WATER_ON", "WATER_OFF"]:
                print("Sending command to Board 1:", cmd)
                e.send(BOARD_1_MAC, ujson.dumps({"cmd": cmd}))
```

```
res.close()
except Exception as err:
    print("Error polling command from EC2:", err)

last_poll_time = time.time()

time.sleep_ms(100)
```

ฝั่ง Server บน EC2 จะใช้ **Flask** ทำหน้าที่เปิด Web API รับข้อมูล บันทึกลงไฟล์ .csv และดูแล Queue คำสั่งรดน้ำ

สคริปต์ Server (app.py)

```
# app.py - รับบน AWS EC2
from flask import Flask, request, jsonify
import csv
import os
from datetime import datetime

app = Flask(__name__)

CSV_FILE = 'sensor_data.csv'
pending_command = "NONE"

# สร้างไฟล์ CSV และ header หากยังไม่มี
if not os.path.exists(CSV_FILE):
    with open(CSV_FILE, mode='w', newline="", encoding='utf-8') as f:
        writer = csv.writer(f)
        writer.writerow(['timestamp', 'temperature', 'humidity'])
```

```
@app.route('/api/data', methods=['POST'])
def receive_data():
    """ รับค่าอุณหภูมิความชื้นจากบอร์ด 2 และเขียนลงไฟล์ CSV """
    data = request.get_json()
    if not data or 'temp' not in data or 'hum' not in data:
        return jsonify({'error': 'Invalid payload'}), 400

    now = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    temp = data['temp']
    hum = data['hum']

    # บันทึกลง CSV
    with open(CSV_FILE, mode='a', newline="", encoding='utf-8') as f:
        writer = csv.writer(f)
        writer.writerow([now, temp, hum])

    print(f"[{now}] Saved: Temp={temp}°C, Hum={hum}%")
    return jsonify({'status': 'success'}), 200
```

```
@app.route('/api/command', methods=['GET'])
def get_command():
    """ ให้ออร์ด 2 มาคอยถามว่ามีคำสั่งรดน้ำหรือไม่ """
    global pending_command
    cmd = pending_command
    pending_command = "NONE" # เคลียร์คำสั่งหลังอ่านแล้ว
    return jsonify({'cmd': cmd}), 200

@app.route('/api/trigger', methods=['POST'])
def trigger_watering():
    """ API สำหรับสคริปต์ภายนอกหรือ Cron job ในการสั่งรดน้ำ """
    global pending_command
    data = request.get_json() or {}
    action = data.get('action', 'WATER_ON')

    pending_command = action
    print(f"Triggered command set to: {pending_command}")
    return jsonify({'status': 'command queued', 'cmd': pending_command}), 200
```

```
if __name__ == '__main__':  
    # $uuu Port 5000  
    app.run(host='0.0.0.0', port=5000)
```

สคริปต์เช็คเงื่อนไขและสั่งรดน้ำ (trigger_water.py)

สคริปต์นี้จะอ่านแถวล่าสุดจากไฟล์ sensor_data.csv เพื่อตรวจสอบความชื้น หากความชื้นต่ำกว่าเกณฑ์ (ฝนไม่ตก) จะสั่งรดน้ำเป็นเวลาตามที่กำหนด แล้วสั่งปิดน้ำ

```
# trigger_water.py - รันผ่าน Cron job บน EC2
```

```
import csv
```

```
import time
```

```
import requests
```

```
CSV_FILE = 'sensor_data.csv'
```

```
EC2_API_URL = 'http://127.0.0.1:5000/api/trigger'
```

```
# กำหนดเกณฑ์ความชื้น (%) ถ้าสูงกว่านี้ถือว่าฝนตก/ดินชื้น ไม่ต้องรดน้ำ
```

```
HUMIDITY_THRESHOLD = 80.0
```

```
WATERING_DURATION_SEC = 60 # ระยะเวลารดน้ำ (เช่น 60 วินาที)
```

```
def get_latest_humidity():
    try:
        with open(CSV_FILE, mode='r', encoding='utf-8') as f:
            reader = list(csv.reader(f))
            if len(reader) > 1: # มีข้อมูลอย่างน้อย 1 แถว (ไม่นับ header)
                latest_row = reader[-1]
                return float(latest_row[2]) # คอลัมน์ humidity
    except Exception as e:
        print("Error reading CSV:", e)
    return None

def main():
    hum = get_latest_humidity()
    if hum is None:
        print("No humidity data available. Skipping watering.")
        return

    print(f"Current humidity: {hum}%")
```

```
if hum >= HUMIDITY_THRESHOLD:
    print(f"Humidity is high ({hum}% >= {HUMIDITY_THRESHOLD}%). Skipping watering
    due to rain/high moisture.")
else:
    print("Humidity is low. Triggering watering sequence...")

    # 1. สั่งเปิดน้ำ
    requests.post(EC2_API_URL, json={'action': 'WATER_ON'})

    # 2. รอน้ำทำงานตามเวลาที่กำหนด
    time.sleep(WATERING_DURATION_SEC)

    # 3. สั่งปิดน้ำ
    requests.post(EC2_API_URL, json={'action': 'WATER_OFF'})
    print("Watering sequence complete.")

if __name__ == '__main__':
    main()
```

ติดตั้ง flask
pip3 install flask requests

อย่าลืมเปิด Inbound Rule รับ Traffic พอร์ต 5000 (Custom TCP) เพื่อให้บอร์ด 2 ส่งข้อมูลเข้ามาได้

ตั้งค่า Cron Job บน EC2 เพื่อสั่งรดน้ำตามเวลา : พิมพ์ crontab -e บน EC2 แล้วใส่บรรทัดนี้ลงไป (ตัวอย่าง: สั่งรดน้ำทุกวัน เวลา 07:00 น. และ 17:00 น.):

```
0 7,17 * * * /usr/bin/python3 /path/to/trigger_water.py >> /path/to/watering.log  
2>&1
```

รัน Flask App แบบ
Background:

```
nohup python3 app.py &
```

สรุป

